

MecâniQA Automotive Tech — OAT 1: Fundações em Nuvem e Orquestração

Autores: Igor Sena Hagge, Ian da Silva Borges, Jonatan de Souza Ferreira, Laio Henrique Pereira da Silva

Agenda

O objetivo dessa apresentação é demonstrar a containerização, provisionamento como código e orquestração do SaaS MecâniQA.

1. Contexto & Desafio:

Gargalos de infraestrutura local legada.

2. Empacotamento (Docker):

Multi-stage build da API
Java, MySQL e Redis.

3. Orquestração Local (Compose):

Comunicação via DNS interno e persistência.

4. Infraestrutura como Código (Terraform):

Provisionamento declarativo e gestão de estado.

5. Camada de Controle (Kubernetes & K9s):

StatefulSet, Deployments e observabilidade.

Camada Virtual — Empacotamento com Docker

API Java (java-api): Multi-stage build com compilação via Gradle (amazoncorretto:21) e imagem final enxuta em Alpine (amazoncorretto:21-alpine). Segurança: Execução com usuário sem privilégios (appuser:appgroup), reduzindo a superfície de ataque no host. Healthcheck: Probe configurada via wget na porta 8080 e entrypoint padrão java -jar app.jar. MySQL & Redis: Imagens oficiais (mysql:8.0 e redis:7-alpine) desacopladas via variáveis de ambiente.

```
user@host: ~ — docker images
```

TAG	IMAGE ID	CREATED	SIZE
optimized	a1b2c3d4e5f6	2 minutes ago	195MB
8.0	b2c3d4e5f6a1	5 days ago	632MB
7-alpine	c3d4e5f6a1b2	2 weeks ago	32.5MB
21	e5f6a1b2c3d4	3 weeks ago	505MB

Orquestração Local & Persistência (Docker Compose)

Conteúdo:

Isolamento e DNS Interno: Comunicação resolvida por nomes de serviço (jdbc:mysql://mysql:3306/myapp e redis:6379).

Mapeamento de Portas: API em 8081:8080, MySQL em 3306:3306 e Redis em 6379:6379.

Persistência Estruturada: Volumes nomeados dedicados (mysql-data:/var/lib/mysql e redis-data:/data).

Ordem de Inicialização: Parâmetro depends_on exigindo condição de saúde (service_healthy) antes da inicialização da API Java.

```
user@host: ~/projeto - docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
java-api	my-corretto-app:optimized	"java -jar app.jar"	api	5 minutes ago	Up 5 minutes (healthy)	0.0.0.0:8081->8080/tcp
mysql	mysql:8.0	"docker-entrypoint.s..."	mysql	5 minutes ago	Up 5 minutes (healthy)	0.0.0.0:3306->3306/tcp, 33060/tcp
redis	redis:7-alpine	"docker-entrypoint.s..."	redis	5 minutes ago	Up 5 minutes (healthy)	0.0.0.0:6379->6379/tcp

Infraestrutura como Código (Terraform)

Provedor Declarativo: Provedor Docker (kreuzwerker/docker) conectado via Named Pipe (npipe:///./pipe//docker_engine).

Recursos Provisionados (main.tf): Rede isolada (app-network), volumes (mysql-data, redis-data) e ciclo de vida dos 3 containers.

Parametrização & Segurança (variables.tf): Portas, tags e credenciais gerenciadas com a flag sensitive.

Gestão de Estado: Backend local com arquivos .tfstate, garantindo deploys reproduzíveis.

```
user@host: ~/projeto/terraform - terraform apply

$ terraform apply -auto-approve

docker_network.app_network: Creating...
docker_network.app_network: Creation complete after 1s [id=app-network]
docker_volume.mysql_data: Creating...
docker_volume.redis_data: Creating...
docker_volume.redis_data: Creation complete after 1s [id=redis-data]
docker_volume.mysql_data: Creation complete after 1s [id=mysql-data]
docker_container.redis: Creating...
docker_container.mysql: Creating...
docker_container.redis: Creation complete after 2s [id=c3d4e5f6a1]
docker_container.mysql: Creation complete after 3s [id=b2c3d4e5f6]
docker_container.api: Creating...
docker_container.api: Creation complete after 2s [id=a1b2c3d4e5]

Apply complete! Resources: 6 added, 0 changed, 0 destroyed.
```

Camada de Controle — Kubernetes & K9s

Topologia de Workloads:

API (Deployment): 2 réplicas balanceadas
(my-corretto-app:optimized).

MySQL (StatefulSet): 1 réplica com persistência estável via PVC de 5Gi
(mysql-pvc).

Redis (Deployment): 1 réplica com armazenamento efêmero
(emptyDir).

Serviços de Rede (Services): LoadBalancer externo na porta 8080
(API), Headless Service (MySQL) e ClusterIP (Redis).

Observabilidade com K9s: Inspeção em tempo real de logs, sondas
de liveness/readiness e métricas de CPU/Memória.

```
k9s - minikube / app-system

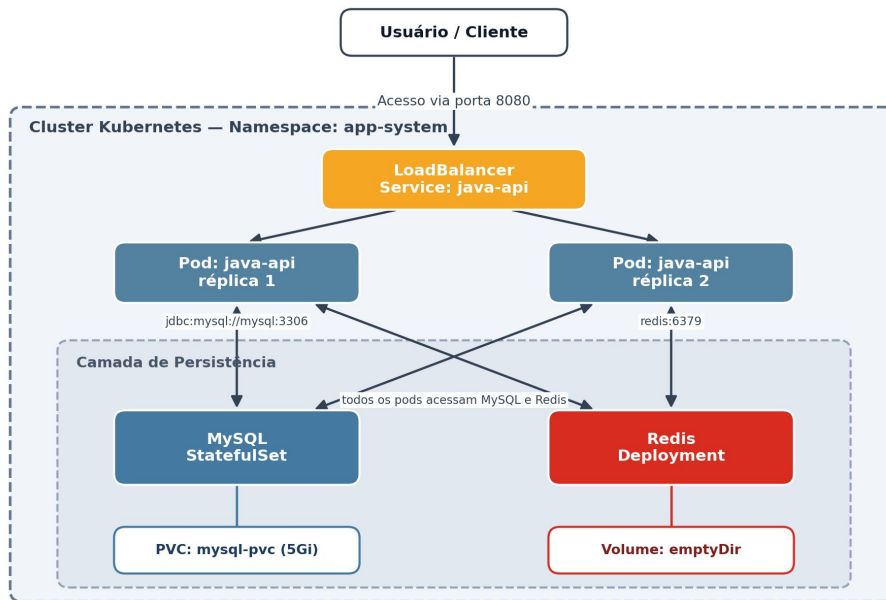
Context: minikube
Cluster: minikube
User: minikube
K9s Rev: v0.32.4

<0> all
<1> default
<2> kube-system
<3> app-system

Pod (app-system) [4]
NAME                                PF  READY  RESTARTS  STATUS  CPU  MEM  CPU/R  CPU/L  MEM/R  MEM/L
java-api-6b8f7d9c4f-2k8m9          ●  1/1    0          Running  2m  210M  250m   500m   512M   1G
java-api-6b8f7d9c4f-l9p2z          ●  1/1    0          Running  3m  215M  250m   500m   512M   1G
mysql-0                             ●  1/1    0          Running  5m  320M  250m   500m   256M   512M
redis-84578b64b9-x4v9b             ●  1/1    0          Running  1m  15M   100m   200m   128M   256M

<pod>
```

Considerações Finais



Resiliência e Recuperação: Eliminação de pontos únicos de falha e auto-cura por políticas de reinício e sondas do K8s.

Padronização de Ambiente: Fim do problema "na minha máquina funciona" com IaC e containerização completa.

Próximos Passos: Evolução da pipeline CI/CD e esteira automatizada para a Sprint 2.

Referências



Documentação Oficial do Docker e Docker Compose: docs.docker.com

Documentação do Kubernetes (StatefulSets & Networking): kubernetes.io/docs

Provedor Docker no Terraform Registry: registry.terraform.io/providers/kreuzwerker/docker

Repositório Oficial do Projeto: [\[github.com/IanBorgess/mecaniQA_oatl_Recife\]](https://github.com/IanBorgess/mecaniQA_oatl_Recife)(https://github.com/IanBorgess/mecaniQA_oatl_Recife)